# IPC

## *Inter Process Communication*

**Professor:** Paulo Roberto de Oliveira
**Disciplina:** Sistemas Operacionais

**Departamento de Informática**
**Universidade Estadual de Maringá**

# Introdução

- A comunicação entre processos é uma necessidade:
  - Num pipeline entre processos, a saída do primeiro. deve ser passada para o segundo, etc.
  - Preferencialmente, o sistema deve ser estruturado e não baseado em interrupções.
- Há 3 questões principais:
  - A passagem de informações de um processo para outro;
  - Evitar que um processo fique no caminho do outro, atrapalhando-se mutuamente;
  - Respeito a dependências.

# Introdução

- Algo Interessante é que esses 3 itens se aplicam a threads também:
  - A passagem de informações é mais simples, já que as threads do mesmo processo compartilham o mesmo espaço de endereçamento;
    - Se estiverem em processos diferentes, o problema cai na comunicação entre processos.
  - Evitar que as threads se atrapalhem mutuamente é o mesmo problema e tem as mesmas soluções;
  - Uma sequência apropriada também pode ser exigida em threads.

# Condições de Corrida

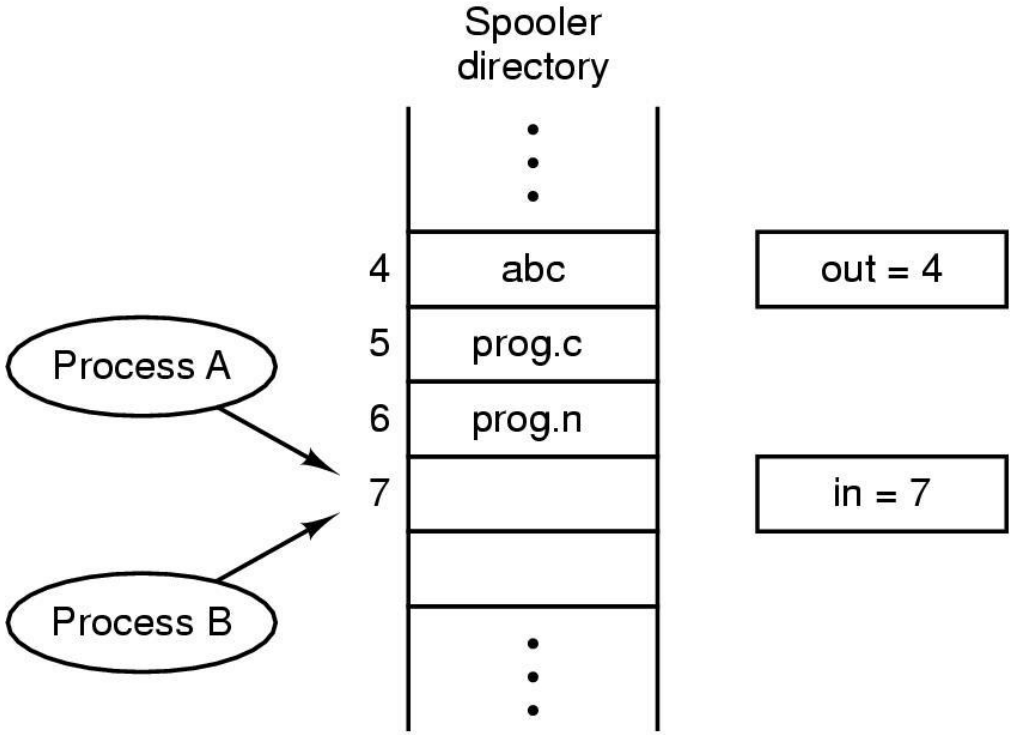
- Processos que trabalham em conjunto podem compartilhar itens, que permitem a leitura e/ou atualização desses.
- Esses itens podem ser mantidos em memória ou em arquivos no disco.
- O uso compartilhado pode levar a interferências perigosas.

- **Observaço:** Spooling ou simplesmente Spool (do acrônimo Simultaneous Peripheral Operations On-line) refere-se a um processo de transferência de dados colocando-os em uma área de trabalho temporária onde outro programa pode acessá-lo para processá-lo em um tempo futuro.

# Condições de Corrida

- **Exemplo:** Suponha que 2 processos decidem imprimir ao mesmo tempo;
- O algoritmo de cada um é o seguinte:

Next_slot_vago = in;
Tabela[next_slot_vago]
    = <nome do arq. A ser impresso>;
Next_slot_vago++;
In = next_slot_vago;



# Condições de Corrida

- Suponha que A comece a processar e executa o 1o. comando;
- Neste momento, o escalonador decide entregar o processador para B;
- B executa o algoritmo todo, deixando um nome de arquivo na entrada 7 e atualizando in para 8;
- Nesse ponto, A retoma sua execução e completa seu algoritmo, deixando outro nome de arquivo na entrada 7 (o arq. de B é perdido);
- In é atualizado de novo para o valor certo – a consistência do sistema está OK, mas B teve seu processamento prejudicado.

# Regiões críticas

- Como se evitam condições de corrida?
- O que deve ser feito é proibir mais de um processo de acessar recursos compartilhados ao mesmo tempo.
- Isso se chama <u>exclusão mútua</u>.
- O problema anterior ocorreu porque o processo B usou ***in*** junto com o proc. A.

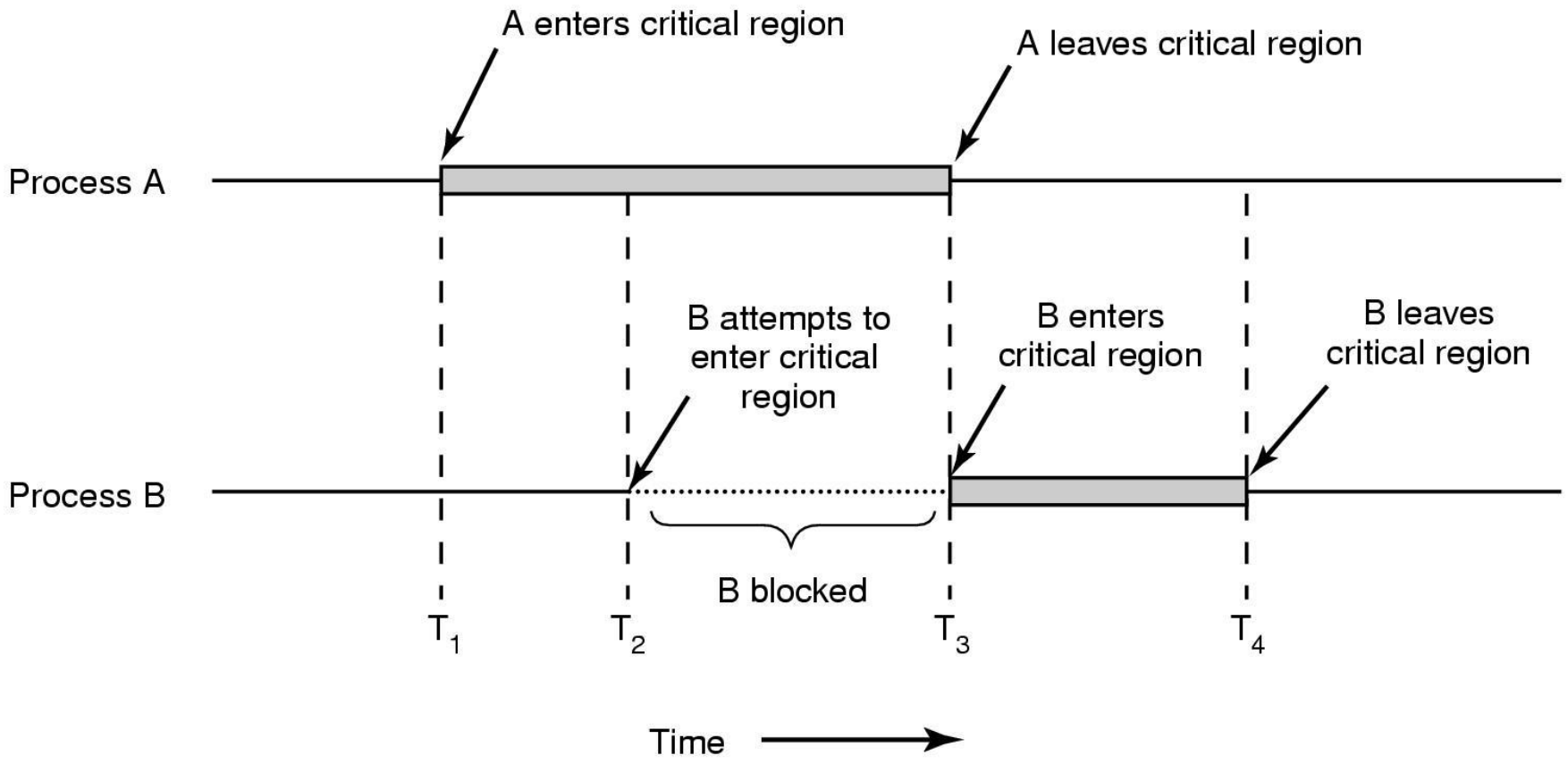
# Regiões críticas

- O problema pode ser formulado de outro modo, também.
- Um processo executa certos trechos de código para realizar cálculos, ou outras atividades. Isso não representa risco de produzir corridas.
- Mas em certas situações, ele desvia para o código que trata dos recursos compartilhados.
- Esse código é chamado de <u>região crítica</u> ou <u>seção crítica</u>.
- Assim, o que precisamos é garantir que 2 ou mais processos não entrem simultaneamente em suas regiões críticas.

# Regiões críticas

- Mesmo essa garantia não é suficiente para evitar condições de corrida. É preciso satisfazer 4 condições para se ter uma boa solução:
  - Dois processos não podem estar simultaneamente dentro de suas regiões críticas;
  - Não se pode presumir condições sobre a velocidade ou número de CPUs;
  - Nenhum processo rodando fora da região crítica pode bloquear outro processo;
  - Nenhum processo deve esperar indefinidamente para entrar em sua região crítica.

# Regiões críticas



# Métodos de Exclusão Mútua

**Desabilitar interrupções:**

- Solução mais simples. O processo desabilita as interrupções antes de entrar na seção crítica e habilita de novo depois de sair.
- Não há interrupções por tempo. O processo não será interrompido, nem mesmo por I/O completado.
- Há 2 problemas principais:
  - O processo pode não habilitar as interrupções novamente – nesse caso, o sistema "morre";
  - Num sistema com múltiplas CPUs, apenas a CPU onde o processo roda é afetada – as demais continuam com interrupções habilitadas e podem acessar o recurso compartilhado.
- Por sua vez, o kernel faz muito uso desse recurso, uma vez que ele possui muitas estruturas cuja manutenção é crítica.

# Métodos de Exclusão Mútua

**Variáveis de bloqueio (LOCK):**

- Segunda tentativa, uma solução por software.
- Um recurso compartilhado é controlado por uma variável única, também compartilhada, começando em ZERO.
- Se um processo quer acessar o recurso, testa a variável. Se for ZERO, passa seu valor para UM e acessa o recurso. Se não, espera até se tornar ZERO.
- Problema?
- A variável de bloqueio já é em si um recurso compartilhado!
- Este método tem o mesmo problema do exemplo do spool. A diferença é que, em vez de acessar a variável ***in*** diretamente, antes o acesso é na variável de lock.

# Métodos de Exclusão Mútua

**<u>Alternação estrita</u>:**

- Este método “funciona”!
- Quem não consegue entrar na região crítica fica testando continuamente até conseguir entrar. Este procedimento chama-se <u>espera ocupada (busy waiting)</u>.
- Neste método, um processo entrega a região crítica para o próximo, independente de ele querer ou não.

```
while (TRUE) {
    while (turn != 0)        /* loop */ ;
    critical_region( );
    turn = 1;
    noncritical_region( );
}
```

(a)

```
while (TRUE) {
    while (turn != 1)        /* loop */ ;
    critical_region( );
    turn = 0;
    noncritical_region( );
}
```

(b)

# Métodos de Exclusão Mútua

**Alternação estrita - Problema:**

- Suponha que (a) tem uma região não crítica muito pequena, enquanto a de (b) é grande.
- Supondo que turn = 0 no início, (a) é o 1o. a usar a região crítica;
- (a) passa turn para 1, executa sua região não crítica e quer entrar na região crítica novamente;
- Como agora turn é 1, (a) entra no loop de espera;
- Com turn 1, (b) entra na região crítica, passa turn para 0 e entra na região não crítica, que é demorada;
- (a) consegue entrar novamente na região crítica, mas agora entrará em loop até que (b) saia de sua região não crítica.
- Esta situação viola a regra 3: O processo (a) está sendo bloqueado por outro fora de sua região crítica.

# Métodos de Exclusão Mútua

**Solução de Peterson (1981):**

- Foi a primeira solução “boa” proposta;
- Consiste em dois procedimentos:
  - enter_region e leave_reagion;
- Antes de usar o recurso deve-se usar o procedimento enter_region;
- Após terminar o uso do recurso deve-se chamar o procedimento leave_reagion;

# Métodos de Exclusão Mútua

## Solução de Peterson (1981):

```
#define FALSE  0
#define TRUE   1
#define N      2                        /* number of processes */

int turn;                               /* whose turn is it? */
int interested[N];                      /* all values initially 0 (FALSE) */

void enter_region(int process);         /* process is 0 or 1 */
{
      int other;                        /* number of the other process */

      other = 1 – process;              /* the opposite of process */
      interested[process] = TRUE;       /* show that you are interested */
      turn = process;                   /* set flag */
      while (turn == process && interested[other] == TRUE) /* null statement */ ;
}

void leave_region(int process)          /* process: who is leaving */
{
      interested[process] = FALSE;      /* indicate departure from critical region */
}
```

# Métodos de Exclusão Mútua

**<u>Instrução TLS</u>:**

- Este método faz uso de um recurso de hardware.
- Alguns processadores possuem uma instrução especial TSL (Test and Set Lock): TSL RX, LOCK
- Esta instrução lê o conteúdo da posição de memória LOCK para o registrador RX e armazena um valor não zero em LOCK.
- A operação toda é indivisível. O processo não pode ser interrompido e LOCK não pode ser acessado até a execução da instrução.

# Métodos de Exclusão Mútua

**Instrução TLS:**

- O sistema de exclusão mútua desse método pode ser visto a seguir:

```
enter_region:
        TSL REGISTER,LOCK                    | copy lock to register and set lock to 1
        CMP REGISTER,#0                      | was lock zero?
        JNE enter_region                     | if it was non zero, lock was set, so loop
        RET | return to caller; critical region entered


leave_region:
        MOVE LOCK,#0                         | store a 0 in lock
        RET | return to caller
```

# Sleep e Wakeup

- A solução de Peterson e TSL são corretas, mas fazem uso de espera ocupada.
- Além de gastar CPU, há efeitos indesejados que podem ocorrer.
- Considere 2 processos, H com alta prioridade e L com baixa prioridade. As regras de escalonamento dizem que H ganha o processador sempre que estiver pronto. Suponha que L está na região crítica e H passa para pronto e quer entrar em sua região crítica. H ganhará o processador e entrará em loop. Após isto L não será mais escalonado.
- Para resolver este problema, foram criadas soluções que bloqueiam processos em vez de permitir que eles “gastem” CPU.
- A mais simples é o par de funções **sleep** e **wakeup**. Sleep é uma função que o processo chama quando deseja se bloquear. Wakeup é uma função que um processo chama para acordar um outro bloqueado.

# O Produtor-Consumidor

- Dois processos compartilham um buffer de tamanho fixo. Um deles é o produtor, que coloca informações no buffer. Outro é o consumidor, que retira as informações.
- É possível montar o modelo com m produtores e n consumidores, mas aqui será considerado apenas um de cada.
- Há duas situações problemáticas:
  - Quando o produtor tenta colocar um item no buffer e ele está cheio;
  - Quando o consumidor tenta retirar um item e o buffer está vazio.

# O Produtor-Consumidor

- Associado ao buffer há uma variável *count* que mantém o número de elementos do buffer. Toda vez que um processo quer inserir ou retirar um item do buffer, deve testar o valor de *count*.
- Se o produtor quer inserir e o buffer está cheio, ele deve dormir.
- Quando o consumidor retira um item e verifica que o buffer estava cheio antes, ele infere que o produtor está dormindo e acorda-o.
- Este método não gasta CPU, mas apresenta problemas. Isso é devido à variável *count*, que é compartilhada e cujo acesso não é controlado.

# O Produtor-Consumidor

```
#define N 100                                   /* number of slots in the buffer */
int count = 0;                                  /* number of items in the buffer */

void producer(void)
{
        int item;

        while (TRUE) {                          /* repeat forever */
                item = produce_item( );         /* generate next item */
                if (count == N) sleep( );       /* if buffer is full, go to sleep */
                insert_item(item);              /* put item in buffer */
                count = count + 1;              /* increment count of items in buffer */
                if (count == 1) wakeup(consumer); /* was buffer empty? */
        }
}
```

# O Produtor-Consumidor

```
void consumer(void)
{
        int item;

        while (TRUE) {                                  /* repeat forever */
                if (count == 0) sleep( );               /* if buffer is empty, got to sleep */
                item = remove_item( );                  /* take item out of buffer */
                count = count – 1;                      /* decrement count of items in buffer */
                if (count == N – 1) wakeup(producer);  /* was buffer full? */
                consume_item(item);                     /* print item */
        }
}
```

# O Produtor-Consumidor

**O Problema:**

- Considere a seguinte sequência de eventos:
  - *Count* é zero;
  - O consumidor vai testar seu valor, mas executa apenas a condição do IF;
  - Nesse momento, o escalonador decide entregar o processador ao produtor;
  - Ele produz um item, insere-o e incrementa *count*;
  - Verifica que count era ZERO e acorda o consumidor;
  - Porém, ele não estava logicamente dormindo;
  - Recuperando o processador, o consumidor chama sleep e dorme;
  - Voltando para o produtor, este produzirá item após item, enchendo o buffer e indo dormir também!
  - **Resultado: DeadLock!**

# O Produtor-Consumidor

**O Problema - Solução:**

- Uma solução para este problema é adicionar um bit de **wakeup-waiting**.
- Se um processo envia um *wakeup* para outro que ainda não está dormindo, o bit é setado. Posteriormente, quando o processo for dormir, com o bit setado ele é imediatamente acordado.
- Apesar de ser uma solução, para um sistema com mais de 2 processos é preciso mais bits. Um por processo não chega a ser suficiente porque um processo pode ser acordado por mais de um processo.
  - Muito Complicado!

# Semáforos

- Em 1965, Dijkstra apresentou uma solução final para o problema.
- Ele sugeriu usar uma variável que seria um contador de *wakeups*, o **semáforo**. O valor ZERO indica que não há *wakeups* pendentes. Um valor positivo indica que 1 ou mais *wakeups* estão pendentes.
- Junto com a variável, são propostas duas operações: *down* e *up*, generalizações de *sleep* e *wakeup*.

# Semáforos

- A operação *down* testa o valor do semáforo. Se ele for maior que ZERO, ele é decrementado (um dos *wakeups* pendentes é “usado”).
- Se for ZERO, o processo é colocado para dormir.
- Tudo isso é feito num procedimento atômico, sem interrupções. Durante a operação, o semáforo não pode mais ser acessado.
- A operação *up* incrementa o valor do semáforo. Se 1 ou mais processos estavam dormindo neste semáforo, eles são acordados um a um pelo escalonador. A operação *down* interrompida é realizada, com o semáforo voltando a ZERO. Agora, um processo a menos está dormindo.

# O Produtor-Consumidor Usando Semáforos

- Primeiro, a implementação de *up* e *down* deve ser baseada em *system calls*. Pois assim, o SO desabilita interrupções para manipular o semáforo.
- Se o sistema possui múltiplas CPUs, cada semáforo deve ser protegido por uma variável de lock, e uma instrução TSL deve ser usada para garantir que apenas uma CPU manipula o semáforo por vez.

# Mutexes

- Se não há a necessidade de usar um semáforo como contador, uma versão simplificada chamada mutex pode ser usada.
- Um mutex é um semáforo binário (só pode assumir 1 ou 0).
- Duas funções são usadas com um mutex:
- Mutex_lock: chamada quando um processo pretende entrar em sua região crítica;
- Mutex_unlock: chamada quando o processo sai da região crítica.

# O Produtor-Consumidor Usando Semáforos Mutexes

```
#define N 100                          /* number of slots in the buffer */
typedef int semaphore;                 /* semaphores are a special kind of int */
semaphore mutex = 1;                   /* controls access to critical region */
semaphore empty = N;                   /* counts empty buffer slots */
semaphore full = 0;                    /* counts full buffer slots */

void producer(void)
{
    int item;

    while (TRUE) {                     /* TRUE is the constant 1 */
        item = produce_item( );        /* generate something to put in buffer */
        down(&empty);                  /* decrement empty count */
        down(&mutex);                  /* enter critical region */
        insert_item(item);             /* put new item in buffer */
        up(&mutex);                    /* leave critical region */
        up(&full);                     /* increment count of full slots */
    }
}
```

# O Produtor-Consumidor Usando Semáforos e Mutexes

```
void consumer(void)
{
        int item;

        while (TRUE) {                          /* infinite loop */
                down(&full);                    /* decrement full count */
                down(&mutex);                   /* enter critical region */
                item = remove_item( );          /* take item from buffer */
                up(&mutex);                     /* leave critical region */
                up(&empty);                     /* increment count of empty slots */
                consume_item(item);             /* do something with the item */
        }
}
```

# Monitores

- O uso de semáforos parece simples, mas há problemas técnicos interessantes.
- Considere a solução do produtor-consumidor. Se em qualquer um dos processos a ordem dos downs for invertida, o processo bloqueia e retém o controle da região crítica, bloqueando também o outro **(deadlock)**.
- Para evitar esse problema, Hoare e Brinch Hansen (1974 e 1975) fizeram a proposta de um sistema de sincronismo de alto nível, **o monitor**.

# Monitores

- Um monitor é um conjunto especial de procedimentos, variáveis e estruturas de dados, agrupados em um módulo especial ou pacote.
- Os processos podem chamar qualquer procedimento do monitor, mas não podem acessar suas variáveis internas diretamente.
- O compilador garante que apenas um processo estará ativo no monitor em um certo momento. Qualquer outro que chame um procedimento do monitor neste ponto será bloqueado.
- Uma linguagem atual que implementa monitores é o **Java**. A adição da palavra-chave **synchronized** a um método garante que só uma thread pode chamar esse método em um determinado instante.

# O Produtor-Consumidor Usando Monitores

```
static class our_monitor {                  // this is a monitor
    private int buffer[ ] = new int[N];
    private int count = 0, lo = 0, hi = 0; // counters and indices
    public synchronized void insert(int val) {
        if (count == N) go_to_sleep( );   // if the buffer is full, go to sleep
        buffer [hi] = val;                // insert an item into the buffer
        hi = (hi + 1) % N;                // slot to place next item in
        count = count + 1;                // one more item in the buffer now
        if (count == 1) notify( );        // if consumer was sleeping, wake it up
    }
    public synchronized int remove( ) {
        int val;
        if (count == 0) go_to_sleep( );   // if the buffer is empty, go to sleep
        val = buffer [lo];                // fetch an item from the buffer
        lo = (lo + 1) % N;                // slot to fetch next item from
        count = count – 1;                // one few items in the buffer
        if (count == N – 1) notify( );    // if producer was sleeping, wake it up
        return val;
    }
    private void go_to_sleep( ) { try{wait( );} catch(InterruptedException exc) {};}
}
}
```

# O Produtor-Consumidor Usando Monitores

```
public class ProducerConsumer {
    static final int N = 100;                    // constant giving the buffer size
    static producer p = new producer( );         // instantiate a new producer thread
    static consumer c = new consumer( );// instantiate a new consumer thread
    static our_monitor mon = new our_monitor( );  // instantiate a new monitor
    public static void main(String args[ ]) {
        p.start( );                              // start the producer thread
        c.start( );                              // start the consumer thread
    }
    static class producer extends Thread {
        public void run( )  {                    // run method contains the thread code
            int item;
            while (true) {                       // producer loop
                item = produce_item( );
                mon.insert(item);
            }
        }
        private int produce_item( ) { ... }      // actually produce
    }
```

```
static class consumer extends Thread {
    public void run( )  {                        run method contains the thread code
        int item;
        while (true) {                           // consumer loop
            item = mon.remove( );
            consume_item (item);
        }
    }
    private void consume_item(int item) { ... }     // actually consume
}
```

# Passagem de Mensagem

- Semáforos só funcionam porque há memória compartilhada entre os processos.
- Em sistemas com múltiplas CPUs, uma instrução TSL pode proteger o semáforo.
- Monitores são restritos porque nem todas as linguagens podem implementar esse recurso.
- Como fazer em Sistemas Distribuídos, onde a memória dos sistemas é distribuída (cada um possui a sua, sem comunicação entre elas)?
- A solução é o uso de **passagem de mensagens**.
- Esse método usa duas funções, **send** e **receive**.

# Passagem de Mensagem

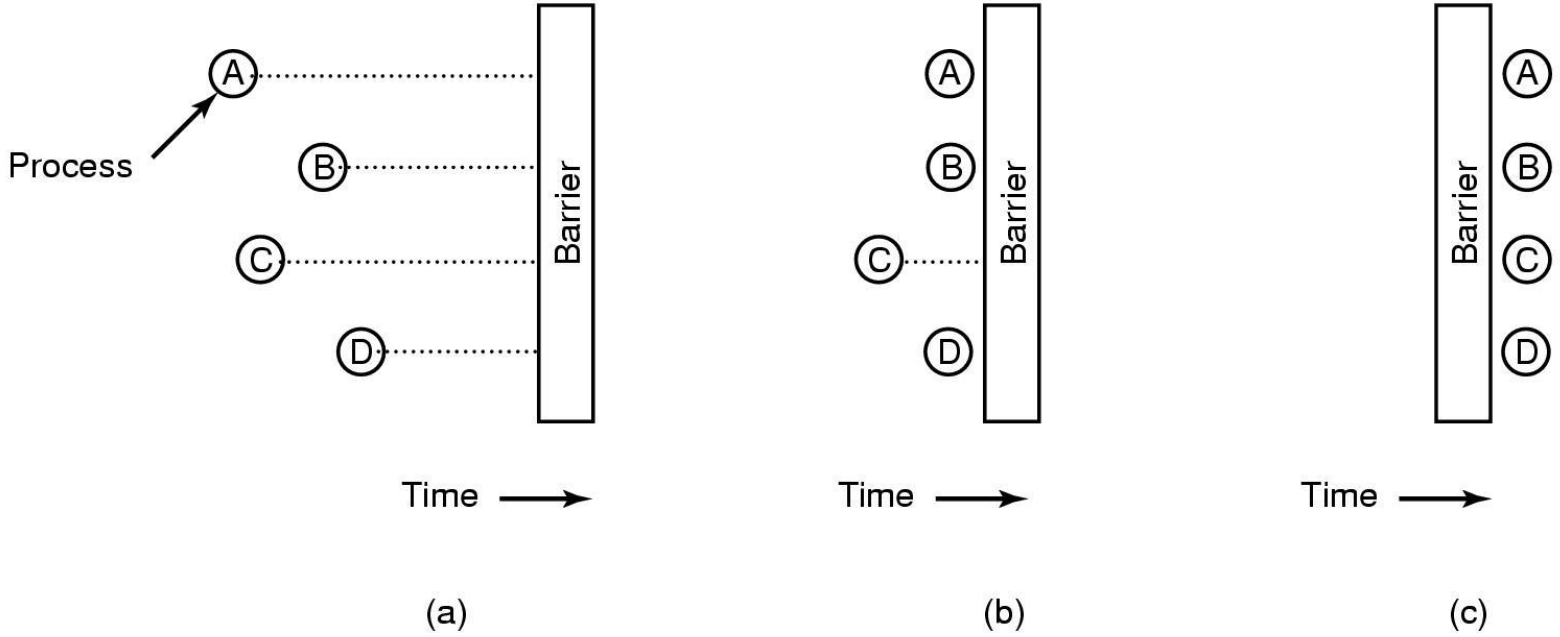
- Essas funções podem ser implementadas em bibliotecas, portanto podem ser acrescentadas a qualquer linguagem sem problemas.
- **Send(destino, $msg)** envia uma msg para um destino qualquer.
- **Receive(fonte, &msg)** recebe uma mensagem de fonte (pode ser *ANY*).
- Há alguns problemas aqui que não existem nos outros métodos. Como as mensagens são enviadas via rede, podem ser extraviadas. Isso exige confirmações (*acknowledgements*), estabelecimentos de relógios para *timeouts* e retransmissões.

# Passagem de Mensagem

- Como um *ack* pode ser extraviado, uma mensagem pode ser enviada duas vezes. Isso exige também o uso de sequências para evitar duplicidade.
- Esse método exige um sistema de identificação dos processos, de forma que send e receive sejam não ambíguos.
- Dependendo do motivo da comunicação, pode-se exigir autenticação das partes envolvidas.
- Entre outros problemas que podem ser ocasionados pelo uso da Rede.

# Barrier

- Uma barreira é um sistema de sincronização para grupos de processos. É voltada para aplicações que funcionam em fases.



# Dúvidas?